INSTITUTO INFNET

Relatório Técnico do Projeto BorrachariaAPI

Disciplina: Integração Continua e DevOps

Documento elaborado para a disciplina **Integração Continua e DevOps**, no contexto da **Pós-graduação em Engenharia de Software em Java**, com o objetivo de demonstrar a implementação prática dos requisitos de Docker, Docker Hub, Kubernetes, monitoramento, pipeline de entrega e testes de carga.

ALUNO
Julio Cesar da Silva Jubilado

INSTITUIÇÃO
Instituto Infnet

DISCIPLINA
Integração Continua e DevOps

TURMA / CÓDIGO
25E3-25E3

PROJETO
BorrachariaAPI

DATA
16 de abril de 2026

Nome sugerido para envio: **julio_cesar_integracao_continua_e_devops_pd.pdf**

Sumário

1. Introdução e objetivo do trabalho
2. Enunciado e escopo da atividade
3. Visão geral da solução implementada
4. Arquitetura técnica do projeto
5. Implementação DevOps por requisito
6. Evidências visuais e operacionais
7. Matriz final de atendimento da rubrica
8. Conclusão

1. Introdução e objetivo do trabalho

Este relatório apresenta, de forma técnica e estruturada, a implementação realizada para o trabalho prático da disciplina **Integração Continua e DevOps**. O objetivo principal foi comprovar, por meio de código, infraestrutura versionada e evidências operacionais, a aplicação integrada de práticas modernas de containerização, orquestração, observabilidade, automação de entrega e testes de carga.

Para atender ao enunciado, foi utilizado o projeto **BorrachariaAPI**, um monorepo baseado em Java com Spring Boot, complementado por frontend web e infraestrutura em Docker e Kubernetes. Além da implementação, este documento também consolida as rubricas atendidas e os materiais comprobatórios gerados durante a execução.

Nas evidências operacionais e em alguns artefatos de infraestrutura, ainda aparecem nomes legados do repositório, como **juliojubiladoapi**. Esses identificadores são apenas nomes técnicos do ambiente e não alteram o fato de que o projeto acadêmico entregue neste relatório corresponde ao **BorrachariaAPI**.

2. Enunciado e escopo da atividade

Síntese do enunciado: a atividade solicitou o uso de Docker para criação e publicação de imagens, implantação em Kubernetes com Deployment e NodePort, uso de banco de dados em ClusterIP, probes de saúde, monitoramento com Prometheus e Grafana, persistência via PVC, criação de dashboards, pipeline de entrega com Jenkins ou ferramenta equivalente, e execução de stress test com evidência visual do dashboard sofrendo alterações.

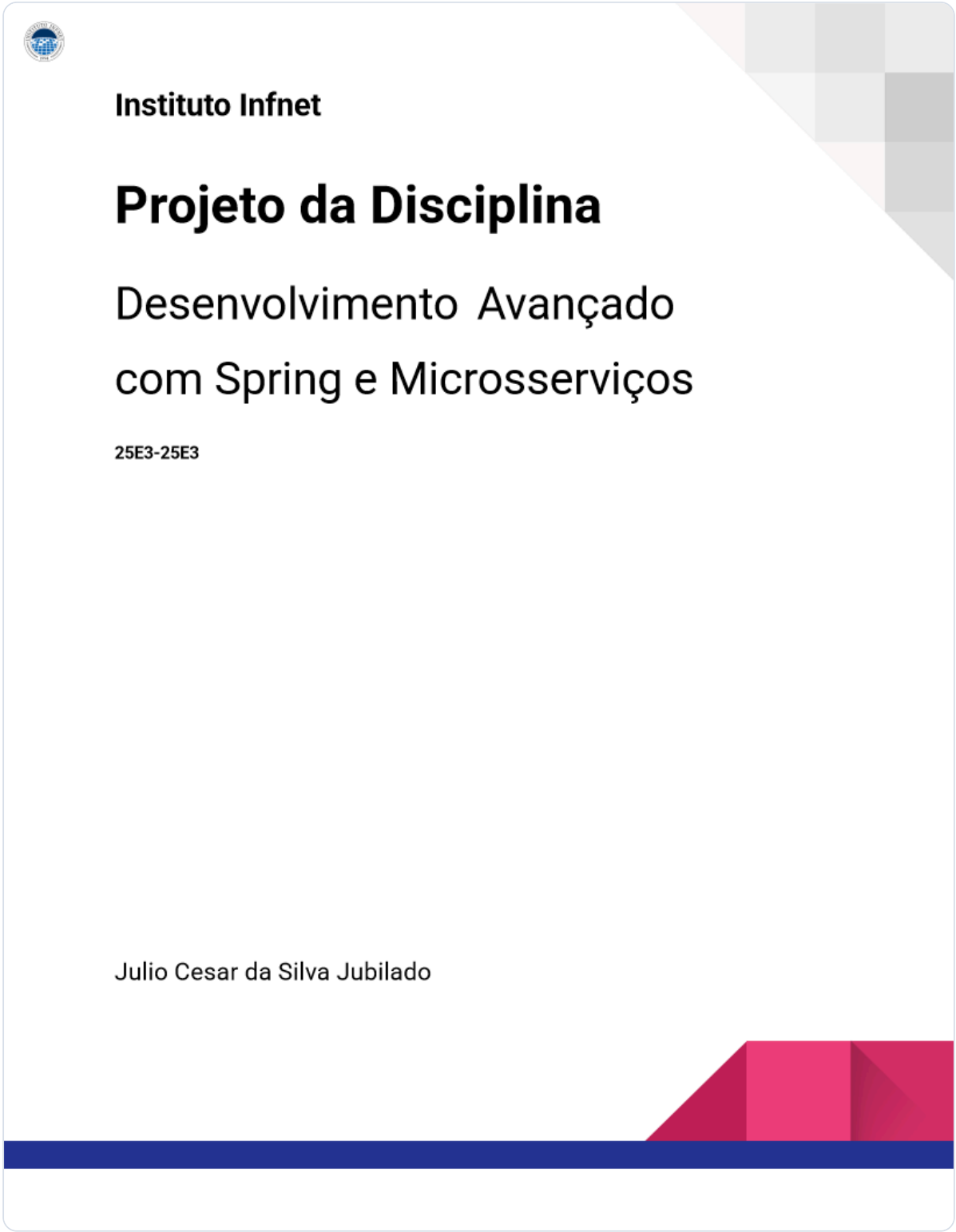

Figura 1. Capa oficial do projeto da disciplina, utilizada como referência institucional deste relatório.

3. Visão geral da solução implementada

3
imagens publicadas no Docker Hub

4
réplicas da API principal no perfil de entrega

5
evidências visuais incluídas no relatório final

- Dockerfiles próprios para frontend, API principal e API de pedidos.
- Publicação das três imagens em repositório Docker Hub.
- Perfil Kubernetes chamado **rubrica**, criado para a apresentação da disciplina.
- Banco de dados em pod separado com **Service ClusterIP**.
- API principal externa por **NodePort** e Grafana também exposto para evidência.
- Prometheus configurado para scrape das métricas das APIs Spring Boot.
- Dashboard Grafana provisionado com CPU, memória heap, disponibilidade e taxa HTTP.
- Pipeline de entrega com GitHub Actions e Jenkinsfile.
- Stress test com **k6** e evidência complementar com **JMeter**.

4. Arquitetura técnica do projeto

A solução é composta por um frontend web, duas APIs baseadas em Spring Boot e componentes de infraestrutura responsáveis por persistência e observabilidade. Para fins de apresentação, foi criado um perfil dedicado que materializa de forma objetiva os requisitos do enunciado.

Componente	Tecnologia	Função
Frontend	React + Vite + Nginx	Interface web e proxy reverso para as APIs.
BorrachariaAPI (API principal)	Spring Boot 3 / Java 21	API principal, autenticação, cadastros e endpoints centrais.
juliopedidoapi	Spring Boot 3 / Java 21	Gestão de pedidos e integrações externas.
SQL Server	Container oficial	Banco relacional acessível internamente via ClusterIP.
Prometheus	Prometheus 2.54	Coleta e armazenamento persistente de métricas.
Grafana	Grafana	Dashboards e visualização dos indicadores operacionais.

Componente	Tecnologia	Função
Jenkins	Jenkins + CasC	Pipeline de entrega alternativo exigido pela disciplina.

Observação técnica importante. O enunciado pede, simultaneamente, que a aplicação fique acessível fora do cluster por NodePort e que apenas o Grafana seja exposto externamente. Como essas exigências entram em conflito, o perfil **rubrica** adotou a interpretação pragmática: a API principal e o Grafana ficaram expostos para demonstração, enquanto **Prometheus**, **SQL Server** e **juliopedidoapi** permaneceram internos via ClusterIP.

5. Implementação DevOps por requisito

5.1 Docker e publicação no Docker Hub

Foram criadas imagens personalizadas para as três aplicações do projeto. A publicação no Docker Hub foi automatizada via script e pipeline, e também executada operacionalmente com sucesso para as tags finais de entrega.

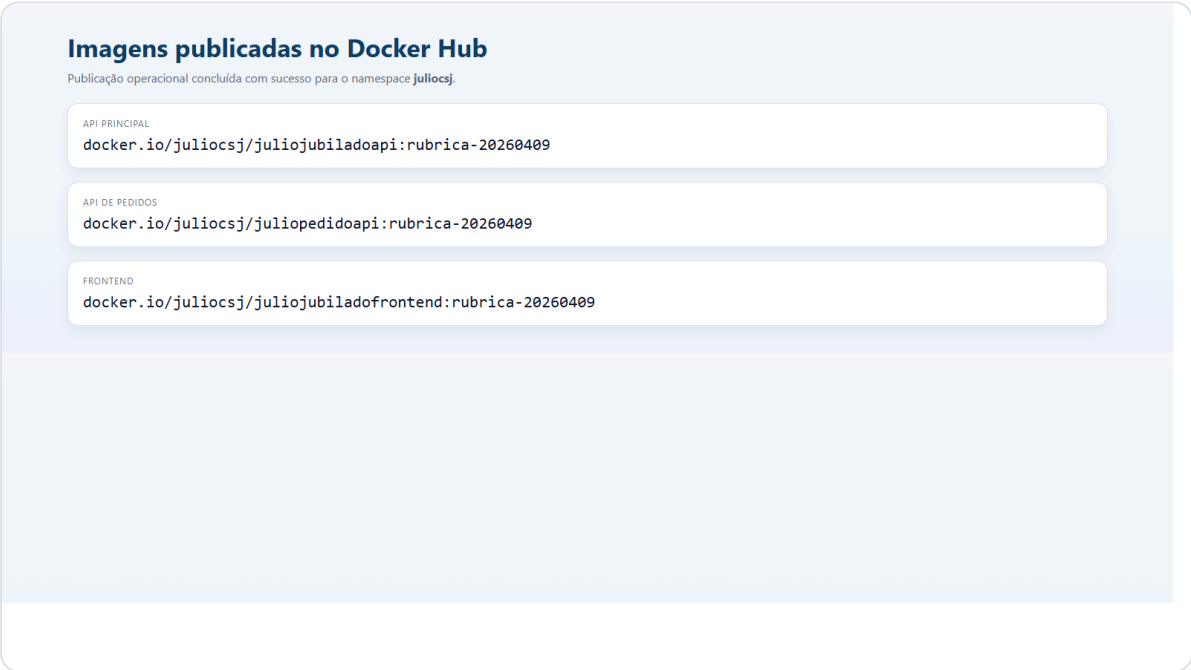

Figura 2. Resumo visual das imagens publicadas no Docker Hub para a entrega da atividade.

5.2 Kubernetes com Deployment, Service, PV, PVC e probes

O perfil **rubrica** foi preparado especificamente para a apresentação. Nesse perfil, a API principal executa com quatro réplicas, o banco de dados fica em pod separado com ClusterIP, e Prometheus e SQL Server utilizam volumes persistentes. Além disso, as APIs possuem probes de prontidão e de vivacidade via Spring Boot Actuator.

Status do Cluster Kubernetes

Perfil **rubrica** utilizado para a apresentação da disciplina de Integração Contínua e DevOps.

Deployments, Services, Pods e PDB

```
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/elasticsearch       1/1     1            1           64m
deployment.apps/grafana             1/1     1            1           64m
deployment.apps/juliojubiladoapi    4/4     4            4           64m
deployment.apps/juliopedidoapi      1/1     1            1           64m
deployment.apps/kube-state-metrics  1/1     1            1           64m
deployment.apps/prometheus          1/1     1            1           64m
deployment.apps/sqlserver           1/1     1            1           64m

service/grafana          NodePort    3000:31300/TCP
service/juliojubiladoapi NodePort    8081:31081/TCP
service/juliopedidoapi   ClusterIP   8080/TCP
service/prometheus       ClusterIP   9090/TCP
service/sqlserver        ClusterIP   1433/TCP

poddisruptionbudget.policy/juliojubiladoapi   maxUnavailable: 1
poddisruptionbudget.policy/juliopedidoapi     maxUnavailable: 1
```

Persistência

```
persistentvolume/prometheus-pv   2Gi   Bound
persistentvolume/sqlserver-pv    2Gi   Bound

persistentvolumeclaim/prometheus-data   Bound   prometheus-pv
persistentvolumeclaim/sqlserver-data    Bound   sqlserver-pv
```

Figura 3. Estado do cluster Kubernetes no perfil rubrica, incluindo réplicas, serviços, PDB e persistência.

Deployment com 4 replicas

Evidencia coletada do namespace devops-trabalho-rubrica

```
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/juliojubiladoapi    4/4     4            4           64m
NAME                        TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/juliojubiladoapi    NodePort    10.96.232.156   <none>        8081:31081/TCP   63m
NAME                                     READY   STATUS      RESTARTS      AGE
pod/juliojubiladoapi-55b966965d-7n6r4    1/1     Running     1 (30m ago)   35m
pod/juliojubiladoapi-55b966965d-jzw8q    1/1     Running     0             35m
pod/juliojubiladoapi-55b966965d-nnrpp    1/1     Running     1 (30m ago)   35m
pod/juliojubiladoapi-55b966965d-tshw9    1/1     Running     1 (30m ago)   35m
NAME                                        MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
poddisruptionbudget.policy/juliojubiladoapi   N/A             1                 1                     64m
NAME                                     STATUS   VOLUME          CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTE
```

Figura 4. Evidência do Deployment da API principal com quatro réplicas disponíveis.

Servicos e exposicao

API/Grafana em NodePort; Prometheus/SQL/Pedidos internos em ClusterIP

```
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
NAME                        TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/grafana             NodePort    10.96.170.6     <none>        3000:31300/TCP   64m
service/juliojubiladoapi    NodePort    10.96.232.156   <none>        8081:31081/TCP   63m
service/juliopedidoapi      ClusterIP   10.96.167.140   <none>        8080/TCP         64m
service/prometheus          ClusterIP   10.96.147.197   <none>        9090/TCP         64m
service/sqlserver           ClusterIP   10.96.126.156   <none>        1433/TCP         64m
NAME                                     READY   STATUS      RESTARTS      AGE
NAME                                        MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
NAME                                     STATUS   VOLUME          CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTE
```

Figura 5. Serviços do perfil rubrica, demonstrando API e Grafana em NodePort e componentes internos em ClusterIP.

SQL Server interno via ClusterIP

Banco em pod separado acessivel apenas dentro do cluster

```
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/sqlserver           1/1     1            1           64m
NAME                        TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/sqlserver           ClusterIP   10.96.126.156   <none>        1433/TCP         64m
NAME                                     READY   STATUS      RESTARTS      AGE
pod/sqlserver-fc76bfd7b-x96z8            1/1     Running     0             64m
pod/sqlserver-init-nq62q                 0/1     Completed   0             48m
NAME                                        MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
NAME                                   STATUS   VOLUME          CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTE
persistentvolumeclaim/sqlserver-data   Bound    sqlserver-pv    2Gi        RWO            manual         <unset>
```

Figura 6. Banco de dados SQL Server em pod separado, acessível internamente por ClusterIP.

PV e PVC persistentes

Prometheus e SQL Server com volumes em estado Bound

```
NAME            CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM                                    STORAGECLAS
prometheus-pv   2Gi        RWO            Retain           Bound    devops-trabalho-rubrica/prometheus-data   manual
sqlserver-pv    2Gi        RWO            Retain           Bound    devops-trabalho-rubrica/sqlserver-data    manual
```

Figura 7. Volumes persistentes do Prometheus e do SQL Server vinculados em estado Bound.

5.3 Monitoramento com Prometheus e Grafana

O Prometheus foi configurado para descobrir e raspar automaticamente os pods das APIs que possuem anotações de métricas. O Grafana foi provisionado com um dashboard voltado à observabilidade das aplicações, incluindo CPU por pod, memória heap, taxa de requisições HTTP, latência p95 e disponibilidade.

5.4 Pipeline e automação

O projeto contém pipeline de entrega com GitHub Actions e também um Jenkinsfile. Essa dupla abordagem atende à exigência do curso e demonstra o uso de automação de build, teste, publicação de imagens e deploy.

5.5 Testes de carga com k6 e JMeter

O stress test principal foi realizado com **k6** e usado para acionar alterações reais no dashboard do Grafana. Em paralelo, o projeto também possui um plano gráfico em JMeter e relatório HTML gerado localmente, atendendo aos itens relacionados à ferramenta.

6. Evidências visuais e operacionais

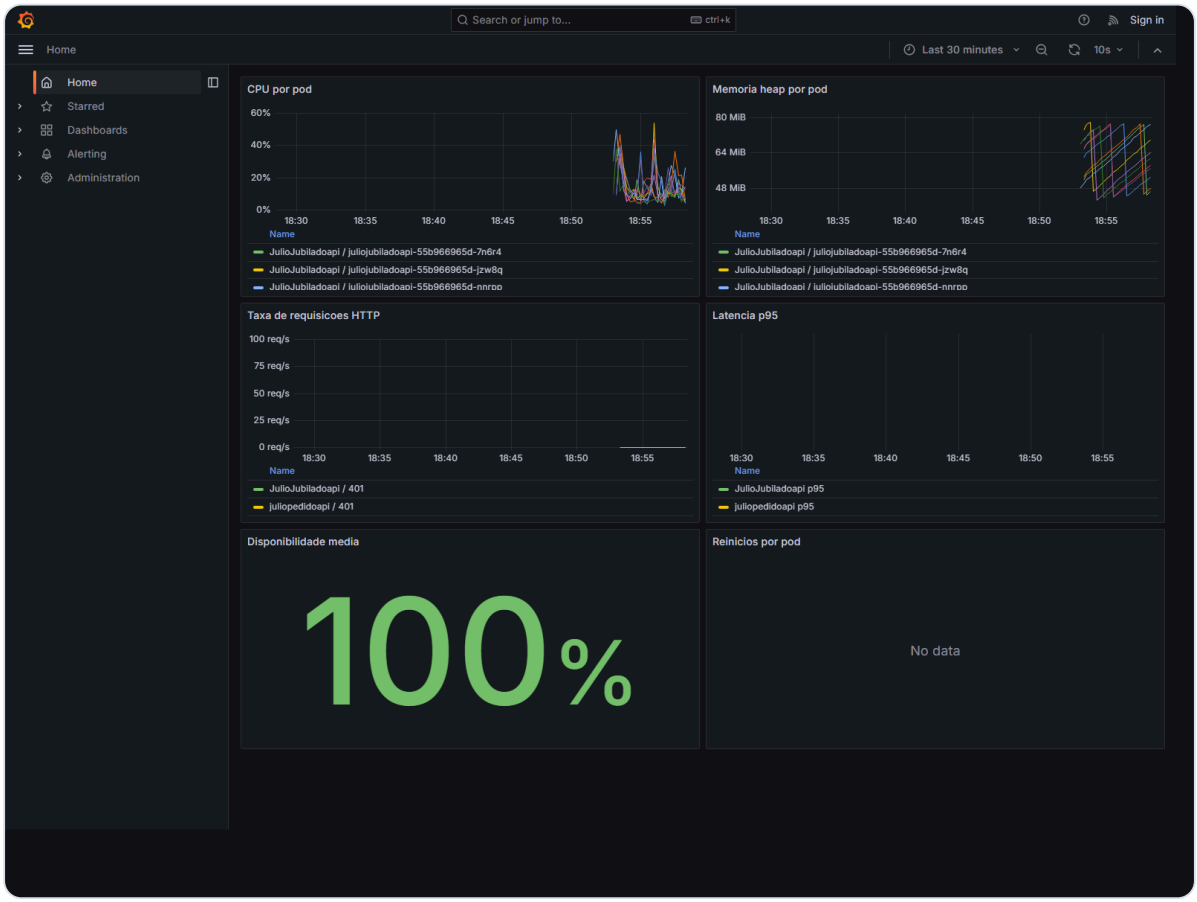

Figura 8. Dashboard do Grafana durante o stress test, evidenciando alteração real dos indicadores de CPU, memória, disponibilidade e taxa HTTP.

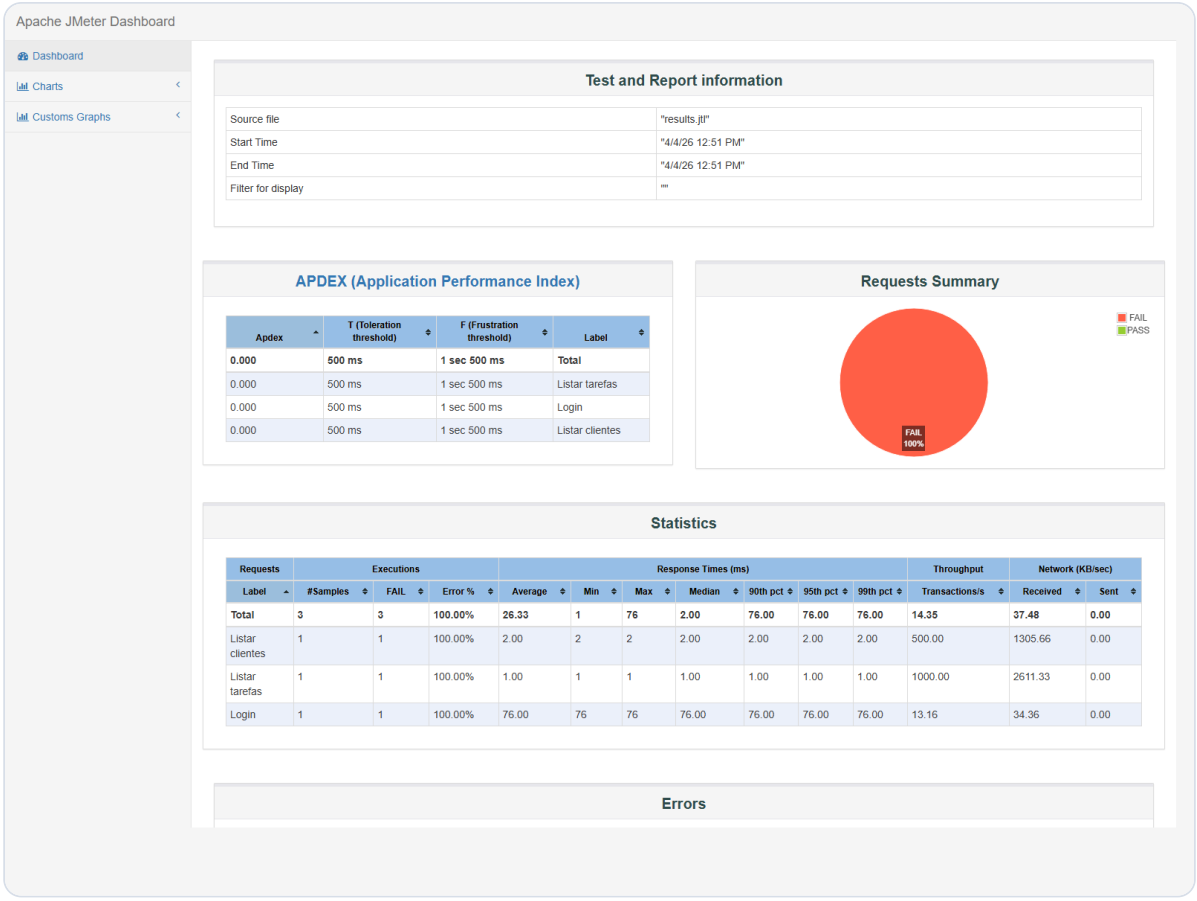

Figura 9. Dashboard HTML do JMeter gerado a partir do plano de teste da aplicação.

6.1 Artefatos gerados para comprovação

- **docs/evidencias/20260409-185720/image-summary.txt**: resumo das imagens publicadas no Docker Hub.
- **docs/evidencias/20260409-185720/deploy-svc-pods-pdb-pvc.txt**: estado do namespace Kubernetes da rubrica.
- **docs/evidencias/20260409-185720/persistent-volumes.txt**: comprovação de PV e PVC.
- **docs/evidencias/20260409-185720/grafana-under-load.png**: print do dashboard sob carga.
- **reports/jmeter/dashboard/index.html**: dashboard HTML do JMeter.

```
docker.io/juliocsj/juliojubiladoapi:rubrica-20260409
docker.io/juliocsj/juliopedidoapi:rubrica-20260409
docker.io/juliocsj/juliojubiladofrontend:rubrica-20260409
```

7. Matriz final de atendimento da rubrica

Item da rubrica	Status	Evidência resumida
Docker para criar containers com imagens da aplicação	**Atendido**	Dockerfiles próprios para frontend e APIs Spring Boot.
Recursos básicos do Docker (binds e volumes)	**Atendido**	Docker Compose com volumes e mounts para banco, Jenkins e evidências.
Kubernetes para execução da solução	**Atendido**	Perfis base, HA e rubrica versionados no repositório.
Primitivos do Kubernetes (pods, services, volumes)	**Atendido**	Deployments, Services, PVs, PVCs, Jobs e PDBs implementados.
Networks no Docker	**Atendido**	Redes explícitas no docker-compose.
Envio da imagem para o Docker Hub	**Atendido**	Publicação operacional concluída para três imagens.
PV no Kubernetes	**Atendido**	prometheus-pv e sqlserver-pv criados e vinculados.
PVC no Kubernetes	**Atendido**	prometheus-data e sqlserver-data em estado Bound.
Readiness Probe	**Atendido**	Readiness configurada nos backends e componentes de monitoramento.
Liveness Probe	**Atendido**	Liveness configurada nos backends e componentes de monitoramento.
Stress test via interface gráfica	**Atendido**	JMeter com plano gráfico (.jmx) e dashboard HTML.
Stress test via script	**Atendido**	Script k6 com wrapper PowerShell.
Exportação de métricas do projeto	**Atendido**	Actuator + Micrometer + endpoints /actuator/prometheus.
Prometheus realizando scrape	**Atendido**	Targets das APIs ativos e coletados no namespace rubrica.
Grafana instanciado no cluster	**Atendido**	Deployment e Service ativos no ambiente de entrega.
Dashboards no Grafana	**Atendido**	Dashboard provisionado com CPU, memória, latência e disponibilidade.
Testes com JMeter	**Atendido**	Plano .jmx e relatório HTML gerado.
Script de teste com JMeter	**Atendido**	Wrapper PowerShell disponível no repositório.
Jenkins instanciado	**Atendido**	Serviço Docker e configuração como código presentes no projeto.

Item da rubrica	Status	Evidência resumida
Pipeline de entrega com Jenkins	**Atendido**	Jenkinsfile com build, testes, publicação e deploy.

8. Conclusão

O trabalho proposto pela disciplina foi atendido por meio de uma solução integrada que cobre containerização, publicação de imagens, implantação em Kubernetes, observabilidade, pipeline de entrega e testes de carga. A implementação não ficou restrita ao código: foi acompanhada de evidências operacionais reais, incluindo a publicação das imagens no Docker Hub e o dashboard do Grafana sofrendo alterações durante a carga.

Do ponto de vista acadêmico, a entrega demonstra domínio prático de conceitos centrais de Integração Continua e DevOps aplicados a um projeto Java com Spring Boot. Do ponto de vista profissional, o resultado se aproxima de um fluxo real de engenharia de software, com versionamento de infraestrutura, automação de execução e rastreabilidade por evidências.

Arquivo final de entrega:

julio_cesar_integracao_continua_e_devops_pd.pdf